

Textures, Lighting, and Shadows

Textures

Textures refer to images applied to surfaces of geometric shapes in three-dimensional scenes. For example, the figure below shows an image of a brick wall mapped to a triangle.



Figure 1. A brick wall texture.

To map a texture to a geometric shape, you need to tell each vertex of the shape which part of the texture it corresponds to. Each vertex should have a **texture coordinate** that specifies what part of the texture image to sample from.

Texture coordinates in the x and y axis range from 0 to 1. The process of retrieving the texture color using texture coordinates is known as **sampling**. Texture coordinates start at (0,0) for the lower-left corner of a texture image to (1,1) for the upper right corner of a texture image. The image below shows how texture coordinates are mapped to the triangle.

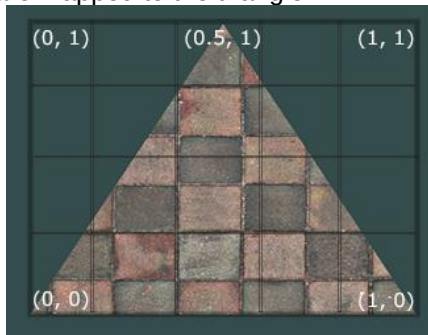


Figure 2. Mapped texture coordinates.

Three (3) texture coordinate points are specified for the triangle. For the bottom-left side of the triangle to correspond with the bottom-left side of the texture, use the (0,0) texture coordinate for the triangle's bottom-left vertex. The same applies to the bottom-right side with a (1,0) texture coordinate. The top of the triangle should correspond with the top-center of the texture image. Hence, (0.5,1.0) is taken as its texture coordinate. You only have to pass three (3) texture coordinates to the vertex shader, which then passes those to the fragment shader. The fragment shader neatly interpolates all the texture coordinates for each fragment.

Texture coordinates usually range from 0 to 1. If you specify coordinates out of this range, the default behavior of OpenGL is to repeat the texture images.

When applying a texture to a surface, the pixels of the texture (**texels**) are usually not precisely the same size as the pixels in the rendered image of the scene. To solve this issue, OpenGL makes use of mipmapping. A **mipmap** is a collection of texture images where each subsequent texture is twice as small compared to the previous one.

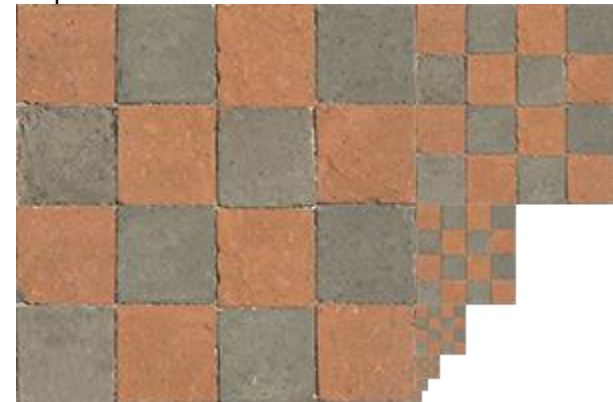


Figure 3. Example of a mipmap.

In OpenGL, a **texture object** is a data structure that stores pixel data from an image. In addition, a texture object stores information about which algorithms to use to determine the colors of the surface the texture is being applied to; this requires some type of computation whenever the texture and the surface are different sizes.

Lighting

Lighting refers to the placement of lights in a scene to achieve a desired effect. It is one of the most important considerations for realistic 3D graphics. The goal is to simulate light sources and how the light they emit interacts with objects in the scene.

Many different types of lighting may be used when rendering an object.

- **Ambient lighting** – affects all points on all geometric surfaces in a scene by the same amount.
- **Diffuse lighting** – represents light that has been scattered and thus will appear lighter or darker in various regions, depending on the angle of the incoming light.
- **Specular lighting** – creates bright spots and highlights on a surface to simulate shininess, which is the tendency of a surface to reflect light.

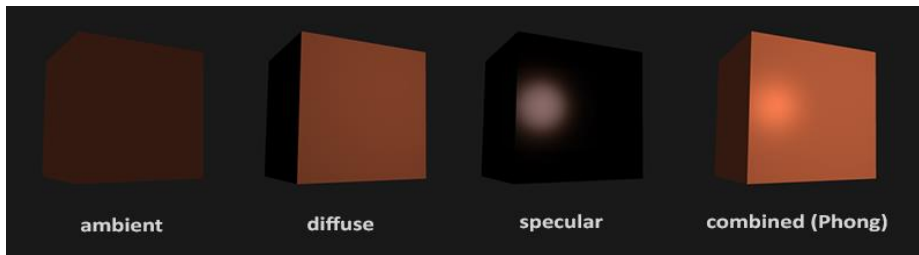


Figure 4. Types of lighting.

An illumination model is a combination of lighting types used to determine the color at each point on a surface. One of the most commonly used illumination models is the **Phong model**, which uses ambient, diffuse, and specular lighting, and is particularly appropriate for reflective or shiny surfaces.

The magnitude of the effect of a light source at a point depends on the angle at which a ray of light meets a surface. This angle is calculated as the angle between two (2) vectors: the direction vector of the light ray and a vector perpendicular to the surface (**normal vector**).

Different types of light objects will be used to simulate different sources of light, each of which emits light rays in different patterns. A **point light** simulates rays of light emitted from a single point in all directions and incorporates attenuation. **Attenuation** is the decrease in intensity as the distance between the light source and the surface increases. A **directional light** simulates a distant light

source such as the sun, in which all the light rays are oriented along the same direction and there is no attenuation.

Shadows

When using lights, the colors on a surface may be brighter or dimmer, depending on the angle at which the light rays meet the surface. This effect is called shading. **Shadows** are areas hidden from the light source. They add great realism to a lit scene and make it easier for viewers to observe and estimate spatial relationships between objects. Shadows give a greater sense of depth to the scene and objects.

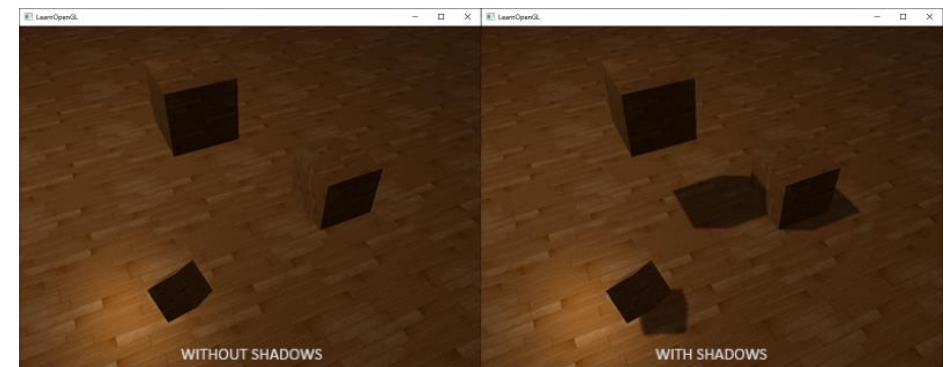


Figure 5. A scene with and without shadows.

With shadows, it becomes more obvious how the objects relate to each other. For instance, the fact that one of the cubes is floating above the others is only noticeable when there are shadows.

The data required for creating shadows will be gathered in a rendering pass called a **shadow pass**. The purpose of it is to render the scene from the position and direction of the light source to determine what points are “visible” to the light. One technique to implement shadows is **shadow mapping**, in which the scene is rendered from the light's point of view. Everything seen from the light's perspective is lit and everything that cannot be seen must be in shadow.

References:

- de Vries, J. (2022). Citing sources. Retrieved from <https://learnopengl.com/>
 Eck, D. (2021). Citing sources. Retrieved from <https://math.hws.edu/>
 Stemkoski, L. & Pascale, M. (2021). *Developing graphics frameworks with Python and OpenGL*. CRC Press.